

SKIMPY

**Alternative to Pandas describe()
for Data Summarization**

Group 3

MEMBER

65050066

กานต์ชนก ตันนารัตน์

65050345

ทักษิณ เชี่ยวกล

67050107

ชัชฎาพงศ์ นุชเทศ

67050238

ธัญรดา แสงพิศาล

67050356

พงษ์พิชญ์ อำนาจกิจเสรี

67050540

ศิวกร วรวัฒนอมรชัย

WHAT IS DATA SUMMARIZATION?

CONCEPT

- The process of converting a massive dataset into a compact description while preserving key information.
- To transform Raw Data into Instant Insights without analyzing every single row.

WHY DO WE NEED

- It is impossible for humans to manually interpret thousands of rows



INTRO - PASDAS DESCRIBE()

WHAT IS IT?

- A function in the pandas library used to generate descriptive statistics.
- By default, it analyzes only numeric columns (int, float).

KEY CAPABILITIES

- automatically calculates count, mean, std, 25%, 50%, 75%, min and max.

HOW TO USE PANDAS DESCRIBE()

DATASET OVERVIEW

Dataset : COVID-19 Patient Records

Code :

```
df=pd.read_csv('/content/covid_dataset.csv')
```

CODE

```
data = pd.DataFrame(df)

print(df.head())
print("-" * 30)
print(data.describe())
```

EXAMPLE RAW DATA

	Age	Gender	Fever	Cough	City	Has_Covid
0	56	Male	102.4	Mild	Mumbai	No
1	19	Female	101.6	Strong	Mumbai	No
2	76	Female	102.0	Mild	Kolkata	No
3	65	Male	99.7	Mild	Bangalore	No
4	25	Female	102.1	Mild	Delhi	Yes

PANDAS DESCRIBE()

	Age	Fever
count	1000.000000	1000.000000
mean	43.461000	100.518400
std	23.267399	1.441339
min	5.000000	98.000000
25%	23.000000	99.200000
50%	43.000000	100.600000
75%	64.000000	101.700000
max	85.000000	103.000000

After executing the code, the system automatically calculates 8 key statistical metrics, including count, mean, std, 25%, 50%, 75%, min and max.

LIMITATIONS OF PANDAS DESCRIBE() IN DATA SUMMARIZATION

01 Limited Support for Non-Numeric Data

- Designed primarily for numeric columns
- `describe(include="all")` provides only basic metrics for non-numeric data

```
print(covid_data.describe(include="all"))
```

	Age	Gender	Fever	Cough	City	Has_Covid
count	1000.000000	1000	1000.000000	1000	1000	1000
unique	NaN	2	NaN	2	4	2
top	NaN	Male	NaN	Mild	Delhi	No
freq	NaN	517	NaN	519	277	518
mean	43.461000	NaN	100.518400	NaN	NaN	NaN
std	23.267399	NaN	1.441339	NaN	NaN	NaN
min	5.000000	NaN	98.000000	NaN	NaN	NaN
25%	23.000000	NaN	99.200000	NaN	NaN	NaN
50%	43.000000	NaN	100.600000	NaN	NaN	NaN
75%	64.000000	NaN	101.700000	NaN	NaN	NaN
max	85.000000	NaN	103.000000	NaN	NaN	NaN

02 Inadequate Missing Data Information

- Requires additional commands to show missing value counts
 - `df.isnull().sum()`

```
print(covid_data.isnull().sum())
```

Age	0
Gender	0
Fever	0
Cough	0
City	0
Has_Covid	0
dtype:	int64

LIMITATIONS OF PANDAS DESCRIBE() IN DATA SUMMARIZATION

03 Lack of Advanced Statistical Insights

- Provides only basic descriptive statistics
- Skewness and kurtosis are supported only for numeric data to describe distribution shape
- No built-in outlier detection or custom aggregations

```
covid_data.describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9])
```

	Age	Fever
count	1000.000000	1000.000000
mean	43.461000	100.518400
std	23.267399	1.441339
min	5.000000	98.000000
10%	11.000000	98.500000
25%	23.000000	99.200000
50%	43.000000	100.600000
75%	64.000000	101.700000
90%	76.000000	102.500000
max	85.000000	103.000000

```
covid_data.select_dtypes(include="number").skew()
```

```
0
Age    0.020616
Fever  -0.042300
dtype: float64
```

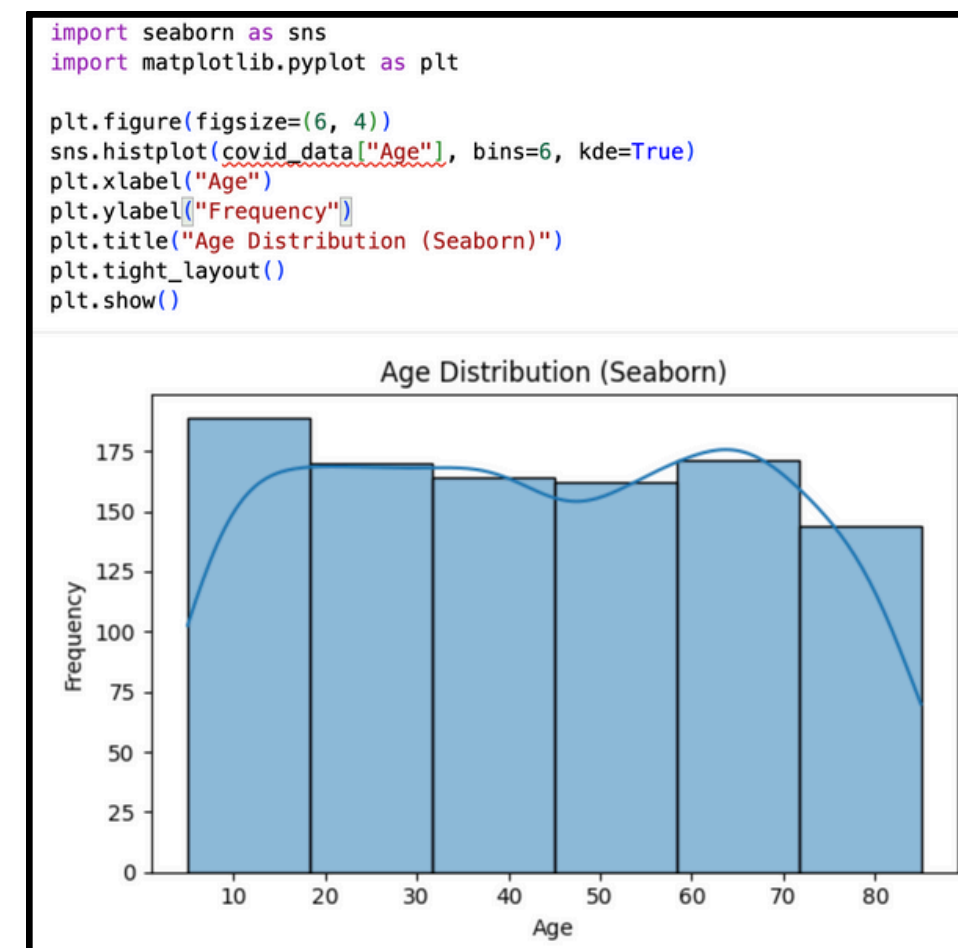
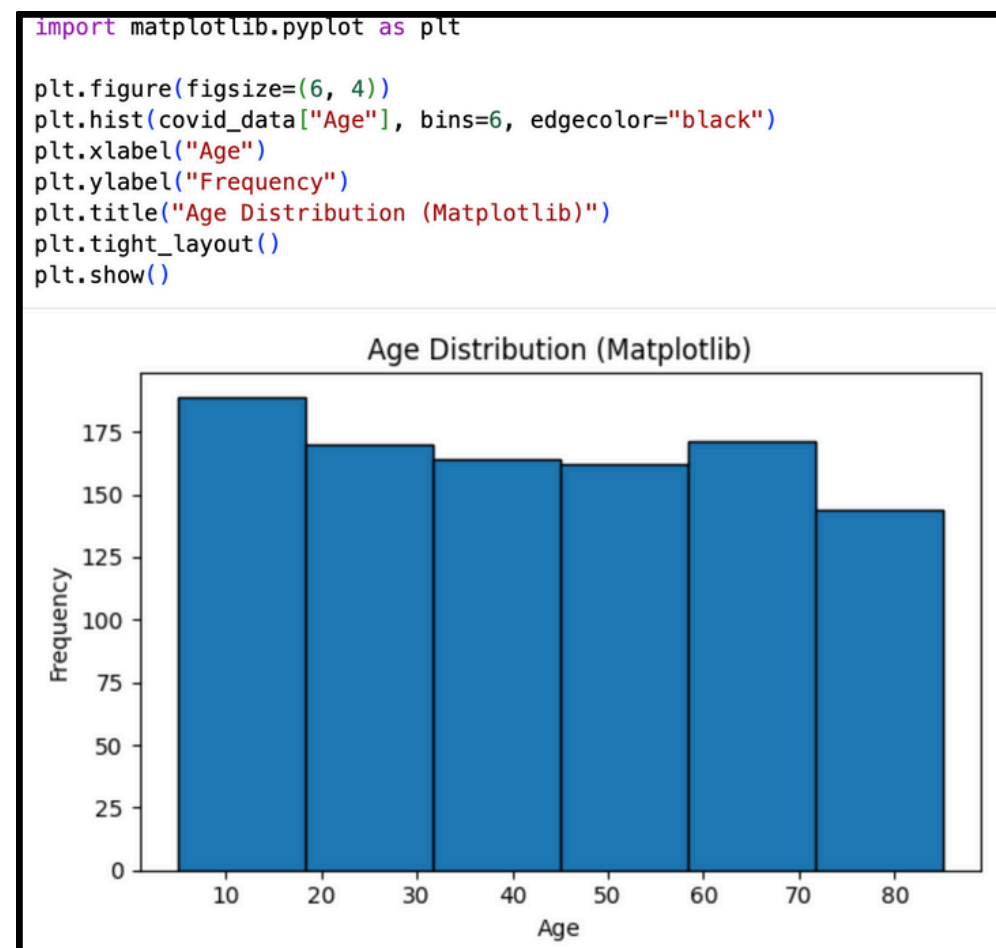
```
covid_data.select_dtypes(include="number").kurt()
```

```
0
Age   -1.211409
Fever -1.192071
dtype: float64
```

LIMITATIONS OF PANDAS DESCRIBE() IN DATA SUMMARIZATION

04 Limited Interpretability and Visualization

- Output is plain text and not presentation-friendly
- Difficult to quickly interpret data quality issues
- Visualization requires external libraries
 - e.g. Matplotlib, Seaborn



SKIMPY

EXPLORATORY DATA ANALYSIS WITH SKIMPY

- => A Lightweight Tool for Quick & Readable Data Summaries
- => Designed to complement Pandas, and focus on data structure and quality assessment
- => Optimized for human readability

DATASET OVERVIEW

- => Dataset : COVID-19 Patient Records (1,000 rows)
- => Mixed Data Types :
 - Numeric : Age, Fever
 - Categorical : Cough, Gender, City
 - Boolean : Has_Covid (Target Variable)

Code : `df = pd.read_csv("covid_data.csv")`

CORE USAGE : ONE-LINE EDA

Key Features

1

One-line Summary : Visualize the entire dataset in a single line of code

2

Auto-detection : Automatically infer data types

3

Rich UI : Displays type-specific summaries in a table with distinct, clearly defined sections

```
skim(df)
```

skimpy summary

Data Summary

Dataframe	Values
Number of rows	1000
Number of columns	6

Data Types

Column Type	Count
string	4
int64	1
float64	1

number

column	NA	NA %	mean	sd	p0	p25	p50	p75	p100	hist
Age	0	0	43.46	23.27	5	23	43	64	85	
Fever	0	0	100.5	1.441	98	99.2	100.6	101.7	103	

string

column	NA	NA %	shortest	longest	min	max	chars per row	words per row	total words
Gender	0	0	Male	Female	Female	Male	4.97	1	1000
Cough	0	0	Mild	Strong	Mild	Strong	4.96	1	1000
City	0	0	Delhi	Bangalore	Bangalore	Mumbai	6.67	1	1000
Has_Covid	0	0	No	Yes	No	Yes	2.48	1	1000

End

skim(df) :

- NA & NA%: Total count and percentage of missing values
- Sparklines (hist): Small-scale bar charts showing the distribution pattern
- Summary Stats: Includes Mean, Median (p50), and Standard Deviation (SD)

```
skim(df[['Age', 'Fever', 'Has_Covid']])
```

skimpy summary

Data Summary

Dataframe	Values
Number of rows	1000
Number of columns	3

Data Types

Column Type	Count
int64	1
float64	1
string	1

number

column	NA	NA %	mean	sd	p0	p25	p50	p75	p100	hist
Age	0	0	43.46	23.27	5	23	43	64	85	
Fever	0	0	100.5	1.441	98	99.2	100.6	101.7	103	

string

column	NA	NA %	shortest	longest	min	max	chars per row	words per row	total words
Has_Covid	0	0	No	Yes	No	Yes	2.48	1	1000

End

```
skim(df[['Age', 'Fever', 'Has_Covid']])
```

```
print("--- ชื่อคอลัมน์ก่อนใช้ clean_columns ---")
print(df.columns.tolist())
print("\n")
```

```
--- ชื่อคอลัมน์ก่อนใช้ clean_columns ---
['First Name', 'AGE (years)', 'Home-Address!', 'phone #', 'Status???']
```

```
df_cleaned = clean_columns(df)
```

```
print("--- ชื่อคอลัมน์หลังใช้ clean_columns ---")
print(df_cleaned.columns.tolist())
```

```
--- ชื่อคอลัมน์หลังใช้ clean_columns ---
['first_name', 'age_years', 'home_address', 'phone_#', 'status']
```

clean_columns(df) :

This function cleans and standardizes messy column names—handling whitespace, inconsistent casing, and special characters—by automatically converting them into snake_case



ADVANTAGE OF SKIMP

1 ONE-LINE SUMMARY

SKIMPY

PANDAS

skim(df)

skimpy summary

Data Summary

Dataframe	Values
Number of rows	1000
Number of columns	6

Data Types

Column Type	Count
string	4
int64	1
float64	1

number

column	NA	NA %	mean	sd	p0	p25	p50	p75	p100	hist
Age	0	0	43.46	23.27	5	23	43	64	85	
Fever	0	0	100.5	1.441	98	99.2	100.6	101.7	103	

string

column	NA	NA %	shortest	longest	min	max	chars per row	words per row	total words
Gender	0	0	Male	Female	Female	Male	4.97	1	1000
Cough	0	0	Mild	Strong	Mild	Strong	4.96	1	1000
City	0	0	Delhi	Bangalore	Bangalore	Mumbai	6.67	1	1000
Has_Covid	0	0	No	Yes	No	Yes	2.48	1	1000

End

df.describe()

	Age	Fever
count	1000.000000	1000.000000
mean	43.461000	100.518400
std	23.267399	1.441339
min	5.000000	98.000000
25%	23.000000	99.200000
50%	43.000000	100.600000
75%	64.000000	101.700000
max	85.000000	103.000000

2 HUMAN READABLE

SKIMPY

skim(df)

skimpy summary

Data Summary

Dataframe	Values
Number of rows	1000
Number of columns	6

Data Types

Column Type	Count
string	4
int64	1
float64	1

number

column	NA	NA %	mean	sd	p0	p25	p50	p75	p100	hist
Age	0	0	43.46	23.27	5	23	43	64	85	
Fever	0	0	100.5	1.441	98	99.2	100.6	101.7	103	

string

column	NA	NA %	shortest	longest	min	max	chars per row	words per row	total words
Gender	0	0	Male	Female	Female	Male	4.97	1	1000
Cough	0	0	Mild	Strong	Mild	Strong	4.96	1	1000
City	0	0	Delhi	Bangalore	Bangalore	Mumbai	6.67	1	1000
Has_Covid	0	0	No	Yes	No	Yes	2.48	1	1000

End

Skimpy-Style EDA with Pandas

- DataFrame.shape: use for get a size of row and column.
- DataFrame.describe(include='all'): use for show stat value all catagory.
- DataFrame.isnull().sum(): use for show missing value.
- DataFrame.dtypes: use for get a type of each column.

PANDAS

df.describe(include='all')

	Age	Gender	Fever	Cough	City	Has_Covid
count	1000.000000	1000	1000.000000	1000	1000	1000
unique	NaN	2	NaN	2	4	2
top	NaN	Male	NaN	Mild	Delhi	No
freq	NaN	517	NaN	519	277	518
mean	43.461000	NaN	100.518400	NaN	NaN	NaN
std	23.267399	NaN	1.441339	NaN	NaN	NaN
min	5.000000	NaN	98.000000	NaN	NaN	NaN
25%	23.000000	NaN	99.200000	NaN	NaN	NaN
50%	43.000000	NaN	100.600000	NaN	NaN	NaN
75%	64.000000	NaN	101.700000	NaN	NaN	NaN
max	85.000000	NaN	103.000000	NaN	NaN	NaN

print(f"Row:\t{df.shape[0]}
Column:\t{df.shape[1]}")

Row:	1000
Column:	6

df.isnull().sum()

	0
Age	0
Gender	0
Fever	0
Cough	0
City	0
Has_Covid	0

df.dtypes

	0
Age	int64
Gender	object
Fever	float64
Cough	object
City	object
Has_Covid	object

dtype: int64 dtype: object

2 HUMAN READABLE

SKIMPY

skim(df)

Data Summary		Data Types	
Dataframe	Values	Column Type	Count
Number of rows	1000	string	4
Number of columns	6	int64	1
		float64	1

number										
column	NA	NA %	mean	sd	p0	p25	p50	p75	p100	hist
Age	0	0	43.46	23.27	5	23	43	64	85	
Fever	0	0	100.5	1.441	98	99.2	100.6	101.7	103	

string									
column	NA	NA %	shortest	longest	min	max	chars per row	words per row	total words
Gender	0	0	Male	Female	Female	Male	4.97	1	1000
Cough	0	0	Mild	Strong	Mild	Strong	4.96	1	1000
City	0	0	Delhi	Bangalore	Bangalore	Mumbai	6.67	1	1000
Has_Covid	0	0	No	Yes	No	Yes	2.48	1	1000

End

Skimpy-Style EDA with Pandas

- DataFrame.shape: use for get a size of row and column.
- DataFrame.describe(include='all'): use for show stat value all catagory.
- DataFrame.isnull().sum(): use for show missing value.
- DataFrame.dtypes: use for get a type of each column.

PANDAS

df.describe(include='all')

	Age	Gender	Fever	Cough	City	Has_Covid
count	1000.000000	1000	1000.000000	1000	1000	1000
unique	NaN	2	NaN	2	4	2
top	NaN	Male	NaN	Mild	Delhi	No
freq	NaN	517	NaN	519	277	518
mean	43.461000	NaN	100.518400	NaN	NaN	NaN
std	23.267399	NaN	1.441339	NaN	NaN	NaN
min	5.000000	NaN	98.000000	NaN	NaN	NaN
25%	23.000000	NaN	99.200000	NaN	NaN	NaN
50%	43.000000	NaN	100.600000	NaN	NaN	NaN
75%	64.000000	NaN	101.700000	NaN	NaN	NaN
max	85.000000	NaN	103.000000	NaN	NaN	NaN

print(f"Row:\t{df.shape[0]}
Column:\t{df.shape[1]}")

Row: 1000
Column: 6

df.isnull().sum()

	0
Age	0
Gender	0
Fever	0
Cough	0
City	0
Has_Covid	0

dtype: int64

df.dtypes

	0
Age	int64
Gender	object
Fever	float64
Cough	object
City	object
Has_Covid	object

dtype: object

3 AUTOMATE DETECTION

SKIMPY

skim(df)

Data Summary		Data Types	
Dataframe	Values	Column Type	Count
Number of rows	1000	string	4
Number of columns	6	int64	1
		float64	1

number									
column		mean	sd	p0	p25	p50	p75	p100	hist
Age		43.46	23.27	5	23	43	64	85	
Fever		100.5	1.441	98	99.2	100.6	101.7	103	

string								
column		shortest	longest	min	max	chars per row	words per row	total words
Gender		Male	Female	Female	Male	4.97	1	1000
Cough		Mild	Strong	Mild	Strong	4.96	1	1000
City		Delhi	Bangalore	Bangalore	Mumbai	6.67	1	1000
Has_Covid		No	Yes	No	Yes	2.48	1	1000

End

Auto Detection

Skimpy automatically categorizes summary statistics by data type, providing more comprehensive details than Pandas' describe() function.

PANDAS

df.describe(include='all')

	Age	Gender	Fever	Cough	City	Has_Covid
count	1000.000000	1000	1000.000000	1000	1000	1000
unique	NaN	2	NaN	2	4	2
top	NaN	Male	NaN	Mild	Delhi	No
freq	NaN	517	NaN	519	277	518
mean	43.461000	NaN	100.518400	NaN	NaN	NaN
std	23.267399	NaN	1.441339	NaN	NaN	NaN
min	5.000000	NaN	98.000000	NaN	NaN	NaN
25%	23.000000	NaN	99.200000	NaN	NaN	NaN
50%	43.000000	NaN	100.600000	NaN	NaN	NaN
75%	64.000000	NaN	101.700000	NaN	NaN	NaN
max	85.000000	NaN	103.000000	NaN	NaN	NaN

4 TIME SAVING

```
import timeit

skim_time = timeit.timeit(lambda: skim(df), number=1)
print("Skimpy:", skim_time)

pandas_time = timeit.timeit(
    lambda: (
        df.info(),
        df.describe(include="all"),
        df.isna().sum()
    ),
    number=1
)
print("Pandas:", pandas_time)
```

```
Skimpy: 0.12480365200008237
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Age         1000 non-null   int64
1   Gender      1000 non-null   object
2   Fever       1000 non-null   float64
3   Cough       1000 non-null   object
4   City        1000 non-null   object
5   Has_Covid   1000 non-null   object
dtypes: float64(1), int64(1), object(4)
memory usage: 47.0+ KB
Pandas: 0.02239271600046777
```

Skimpy Vs Pandas about time

Although execution time differences may be marginal, skimpy significantly reduces developer time by minimizing the number of function calls required for exploratory data analysis.

COMPARISON

Writability

- Finding missing value

```
import pandas as pd
from skimpy import skim

data = {
    "Name": ["Alice", None, "Charlie", "David"],
    "Age": [25, None, 35, None],
    "City": ["New York", "Los Angeles", "Chicago", "Houston"],
    "Salary": [70000, 80000, 120000, 90000]
}

df = pd.DataFrame(data)
```

Writability

Pandas' describe() function doesn't detect missing values. You have to write that logic yourself.

```
df.describe(include="all")
```

--NORMAL--

	Name	Age	City	Salary
count	3	2.000000	4	4.000000
unique	3	NaN	4	NaN
top	Alice	NaN	New York	NaN
freq	1	NaN	1	NaN
mean	NaN	30.000000	NaN	90000.000000
std	NaN	7.071068	NaN	21602.468995
min	NaN	25.000000	NaN	70000.000000
25%	NaN	27.500000	NaN	77500.000000
50%	NaN	30.000000	NaN	85000.000000
75%	NaN	32.500000	NaN	97500.000000
max	NaN	35.000000	NaN	120000.000000

```
df.isnull().sum()
```

0

Name 1

Age 2

City 0

Salary 0

Writability

Meanwhile, skimpy provides all the basic information you need in just one function.

```
skim(df)
```

skimpy summary

Data Summary

Dataframe	Values
Number of rows	4
Number of columns	4

Data Types

Column Type	Count
string	2
float64	1
int64	1

number

column	NA	NA %	mean	sd	p0	p25	p50	p75	p100	hist
Age	2	50	30	7.071	25	27.5	30	32.5	35	
Salary	0	0	90000	21600	70000	77500	85000	97500	120000	

string

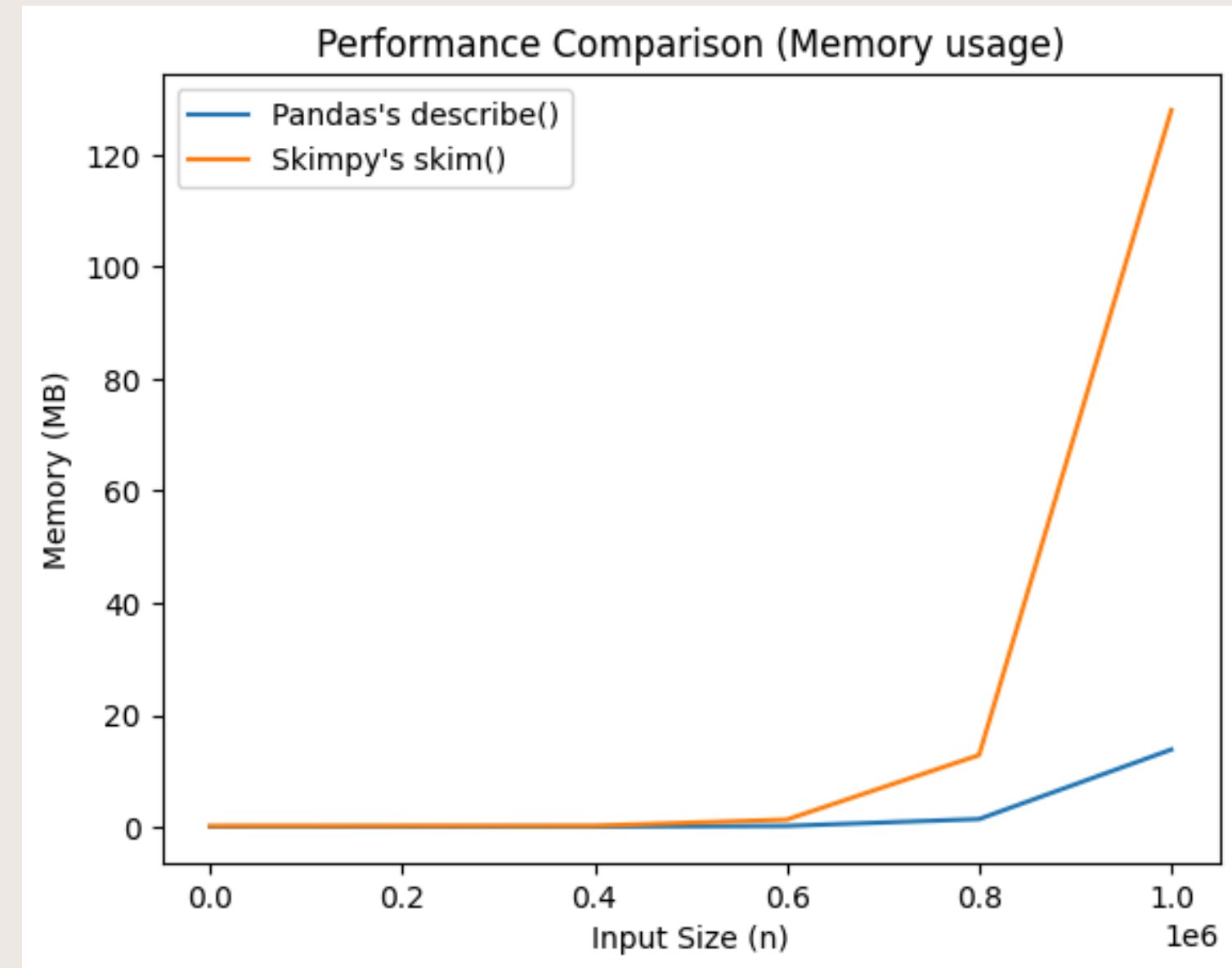
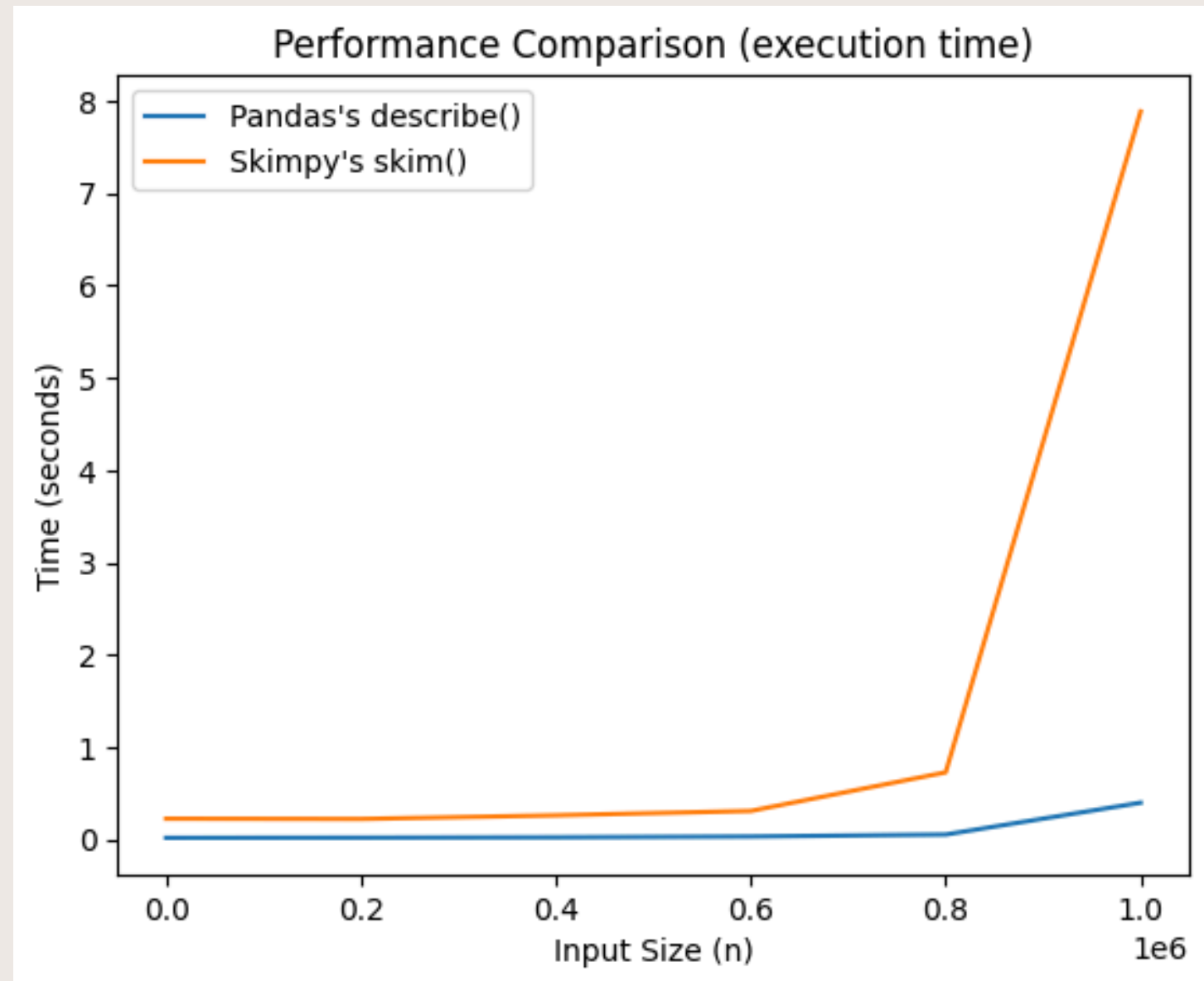
column	NA	NA %	shortest	longest	min	max	chars per row	words per row	total words
Name	1	25	Alice	Charlie	Alice	David	5.67	0.75	3
City	0	0	Chicago	Los Angeles	Chicago	New York	8.25	1.5	6

End

Performance

- We measure the execution time and memory consumption between Pandas's `describe()` and Skimpy's `skim()` on random data set which lead to one million rows. however, note that Skimpy have show us more information that mean it's have processing data more that pandas by nature.

Performance





ALTERNATIVE LIBRARY

YData Profiling

เป็น library ตัวหนึ่งใน python ที่ช่วยในการวิเคราะห์ข้อมูลซึ่งต่างจาก Skimpy ตรงที่ ตัวนี้จะให้ข้อมูลที่ละเอียดกว่าและมีความเป็น interactive

สิ่งที่ได้

1. สรุปภาพรวม (Overview)
2. วิเคราะห์ตัวแปรรายตัว (Variables)
3. การโต้ตอบและความสัมพันธ์ (Interactions & Correlations)
4. วิเคราะห์ข้อมูลที่หายไป (Missing Values)
5. วิเคราะห์ข้อความแบบละเอียด (Text Analysis)

```
[6] !pip install ydata-profiling
```

```
[7] import pandas as pd
from ydata_profiling import ProfileReport
from datetime import datetime
from google.colab import files
import io

# โหลดไฟล์ลง
uploaded = files.upload()

# เอาไฟล์มาใส่ใน DataFrame
file_name = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[file_name]))

# 3. ตั้งชื่อไฟล์รายงาน
current_date = datetime.now().strftime("%Y_%m_%d")
filename = f"report_{file_name}_{current_date}.html"

# สร้าง ProfileReport (คำสั่งของ YData Profiling)
profile = ProfileReport(df, title=f"Report: {file_name}", explorative=True)

# ทำเป็น HTML
profile.to_file(filename)
```

Choose Files covid_dataset.csv
covid_dataset.csv(text/csv) - 32662 bytes, last modified: 12/21/2025 - 100% done
Saving covid_dataset.csv to covid_dataset (2).csv

Summarize dataset: 100% 19/19 [00:01<00:00, 14.24it/s, Completed]

0%| 0/6 [00:00<?, ?it/s]
100%| 6/6 [00:00<00:00, 33.39it/s]

Generate report structure: 100% 1/1 [00:02<00:00, 2.10s/it]

Render HTML: 100% 1/1 [00:00<00:00, 6.40it/s]

Export report to file: 100% 1/1 [00:00<00:00, 41.99it/s]

YData Perfling

Overview

Overview

Alerts 1

Reproduction

Dataset statistics

Number of variables	6
Number of observations	1000
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	6
Duplicate rows (%)	0.6%
Total size in memory	225.8 KiB
Average record size in memory	231.2 B

Variable types

Numeric	2
Categorical	3
Boolean	1

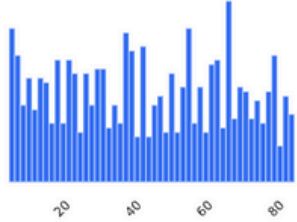
Variables

Select Columns ▾

Age

Real number (R)

Distinct	81	Minimum	5
Distinct (%)	8.1%	Maximum	85
Missing	0	Zeros	0
Missing (%)	0.0%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	43.461	Memory size	7.9 KiB



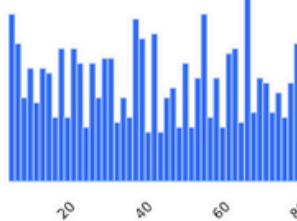
More details

Variables

Select Columns ▾

Select ColumnsAgeGenderFeverCoughCityHas_Covid

Distinct	81	Minimum	5
Distinct (%)	8.1%	Maximum	85
Missing	0	Zeros	0
Missing (%)	0.0%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	43.461	Memory size	7.9 KiB



More details

CONCLUSION



Reference

- Trivedi, A. (2024, November 26). Skimpy: Alternative to Pandas describe() for Data Summarization. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2024/11/skimpy/#h-advantages-of-using-skimpy>
- pandas.DataFrame.describe — pandas 2.1.1 documentation. (n.d.). Pandas.pydata.org. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html#pandas.DataFrame.describe>
- Skimpy. (2018). Github.io. <https://aeturrell.github.io/skimpy/>
- YData Profiling - <https://docs.profiling.ydata.ai/latest/>



THANK YOU

Group 3